Class/Namespace	Functions	Return values
std::unique_lock<*TimedLockable*>	unique_lock(*lockable*,*duration*)unique_lock(*lockable*,*time_point*)	N/A—owns_lock() on the newly constructed object; returns true if the lock was acquired, false otherwise
	try_lock_for(*duration*)try_lock_until(time_point)	bool—true if the lock was acquired, false otherwise
std::future<*ValueType*> or std::shared_future<*ValueType*>	wait_for(*duration*)wait_until(*time_point*)	std::future_status::timeout if the wait timed out, std::future_status::ready if the future is ready, or std::future_status::deferred if the future holds a deferred function that hasn't yet started

Now that I've covered the mechanics of condition variables, futures, promises, and packaged tasks, it's time to look at the wider picture and how they can be used to simplify the synchronization of operations between threads.

4.4. Using synchronization of operations to simplify code

Using the synchronization facilities described so far in this chapter as building blocks allows you to focus on the operations that need synchronizing rather than the mechanics. One way this can help simplify your code is that it accommodates a much more *functional* (in the sense of *functional programming*) approach to programming concurrency. Rather than sharing data directly between threads, each task can be provided with the data it needs, and the result can be disseminated to any other threads that need it through the use of futures.

4.4.1. Functional programming with futures

The term *functional programming* (FP) refers to a style of programming where the result of a function call depends solely on the parameters to that function and doesn't depend on any external state. This is related to the mathematical concept of a function, and it means that if you invoke a function twice with the same parameters, the result is exactly the same. This is a property of many of the mathematical functions in the C++ Standard Library, such as `sin, cos`, and `sqrt`, and simple operations on basic types, such as `3+3, 6*9`, or `1.3/4.7`. A *pure* function doesn't *modify* any external state either; the effects of the function are entirely limited to the return value.

This makes things easy to think about, especially when concurrency is involved, because many of the problems associated with shared memory discussed in chapter 3 disappear. If there are no modifications to shared data, there can be no race conditions and thus no need to protect shared data with mutexes either. This is such a powerful simplification that programming languages such as Haskell,[2] where all functions are pure *by default*, are becoming increasingly popular for programming concurrent systems. Because most things are pure, the *impure* functions that actually *do* modify the shared state stand out all the more, and it's therefore easier to reason about how they fit into the overall structure of the application.

[2] See http://www.haskell.org/.